

Détection de motifs spatio-temporels dans des signaux multivariés

Emma Kerinec

Maitre de stage Grégoire Mercier, encadrant Bastien Padeloup

25 août 2016

Télécom Bretagne



Résumé

Reconnaître des motifs dans des signaux spatio-temporels est un enjeu important pour le traitement de données. Dans ce domaine, la problématique de l'identification d'objets sur des images est connue depuis longtemps.

Cependant on peut désormais étendre le problème en représentant des domaines irréguliers par des graphes et utiliser les outils mathématiques adaptés. On peut ainsi représenter des signaux multivariés et définir une transformée de Fourier très similaire à celle connue usuellement. Cela permet de s'intéresser à des cas plus complexes que les images, des nuages de capteurs par exemple où un même motif à des endroits différents ne sera pas perçu comme identique si les distances entre les capteurs varient.

D'autre part, la classification ou apprentissage supervisé est un domaine regroupant les méthodes pour apprendre à classer des données à partir d'exemples d'apprentissages. Une de ces méthode est nommée machine à vecteurs de support.

Au cours de mon stage, j'ai cherché à utiliser la transformée de Fourier généralisée aux graphes pour être plus performant lors de l'utilisation de cette méthode.

Table des matières

1	Introduction	3
2	Définition et présentation des notions	3
2.1	Signaux sur graphe	3
2.1.1	Qu'est-ce ?	3
2.1.2	Quel est l'intérêt ?	3
2.2	Classification	4
2.2.1	De quoi s'agit-il ?	4
2.2.2	Machines à vecteurs de support :	4
3	Travaux personnels	5
3.1	Idées générales	5
3.2	Premiers tests	5
3.2.1	Inversion de matrices	5
3.2.2	Premières transformées et observations	5
3.3	Premiers apprentissages	6
3.4	Transformation de transformées	7
3.4.1	Modification de valeurs	7
3.4.2	Suppression de valeurs	8
3.5	Apprentissage de transformées simplifiées	9
3.6	Comparaison de temps	10
3.7	Détection d'un motif parmi plusieurs	11
3.8	Divagations	12
4	Conclusion	13

1 Introduction

Récemment des travaux ont proposé d'étendre le traitement de signal classique à des domaines irréguliers tels que les signaux multivariés obtenus par exemple par imagerie cérébrale. Ce traitement s'effectue grâce à la représentation sous forme de signaux sur graphe[1], qui permettent notamment d'étendre les outils classiques du traitement du signal. Un de ces outils est nommé transformée de Fourier, je me suis intéressée au cours de mon stage à son utilité pour détecter des motifs spatio-temporels, en particulier en utilisant une méthode de classification : la méthode des machines à vecteurs de support.

2 Définition et présentation des notions

2.1 Signaux sur graphe

2.1.1 Qu'est-ce ?

Un graphe $G = (V; E)$ est un couple constitué d'un ensemble de points V nommés nœuds (ou sommets) et un ensemble E de couples de ces sommets nommés arêtes, tel que $(u, v) \in E \Leftrightarrow$ une arête part de u vers v . On dit que G est non orienté si et seulement si $(u, v) \in E \Leftrightarrow (v, u) \in E$. Dans le cas contraire, on dira que G est orienté, et les arêtes seront alors nommées *arcs*. On peut également associer à chaque arête du graphe une valeur numérique : on parle dans ce cas de graphe pondéré, et $w : V^2 \rightarrow \mathbb{R}$ est appelée *fonction de pondération*. Si $(i, j) \notin E$ alors $w(i, j) = 0$.

On peut représenter un graphe par sa *matrice d'adjacence* A . A est carrée de taille égale au nombre de sommets du graphe, et $(A)_{i,j} = w(i, j)$.

Un signal sur graphe est une fonction $f : V \rightarrow X$ avec X un ensemble quelconque (généralement $\mathbb{R}$ ou $\mathbb{C}$).

Pour les définitions suivantes, $G = (V, E)$ est un graphe non-orienté pondéré, $n = |V|$, et A sa matrice d'adjacence.

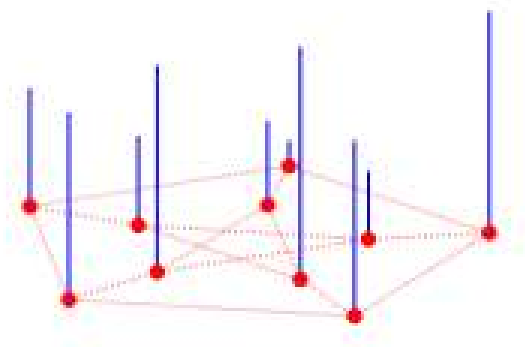


FIGURE 1 – Représentation d'un signal sur graphe. Les points rouges représentent les sommets, les pointillés rouges les arêtes et les lignes verticales bleues le signal.

2.1.2 Quel est l'intérêt ?

Les graphes ont de nombreuses applications dans de multiples domaines tels que le social, l'énergie, les transports. En effet, ils permettent de décrire la structure de nombreux problèmes en représentant les données par les sommets et en utilisant les arêtes pour montrer une relation entre elles (similitude, dépendance...)[2].

On peut par exemple représenter un nuage de capteurs (une disposition irrégulière dans le plan ou l'espace de capteurs) : chaque sommet représente un capteur, chaque arête représente la distance entre les deux capteurs correspondant aux sommets, et le signal correspond à la valeur mesurée par chaque capteur.

Le laplacien de graphe :

Le laplacien de graphe L est une matrice carrée de même taille que A , définie par $L = D - A$, où D est la matrice diagonale des degrés : le i -ème élément correspond à la somme des poids sur les arêtes liées au i -ème sommet.

L est donc une matrice réelle symétrique et de ce fait diagonalisable. Elle possède donc une décomposition dans une base orthonormée de vecteurs propres. Dans la suite de ce rapport, L est le laplacien du graphe G , et on utilisera une base fixée $\mathcal{B} = (u_1, u_2, \dots, u_n)$.

Ces matrices possèdent de nombreuses propriétés liées aux graphes d'origine : notamment la multiplicité de 0 en tant que valeur propre correspond au nombre de composantes connexes dans le graphe.

La transformée de Fourier :

On peut définir la transformée de Fourier d'un graphe de manière analogue à la transformée de Fourier usuelle.

Soit y_i la valeur propre de L associé à u_i , pour f un signal sur graphe à valeur dans $\mathbb{C}$, on définit $\hat{f}$ la fonction transformée par :

$$\hat{f}(y_i) = \langle f, u_i \rangle = \sum_{j=1}^n f(j) \cdot (u_i^*)_j$$

2.2 Classification

2.2.1 De quoi s'agit-il ?

La classification est la science consistant à apprendre à classer des éléments en deux catégories après un apprentissage sur plusieurs exemples. On parle alors d'apprentissage supervisé. On peut par exemple présenter des images de chats en disant "il y a un chat" et d'autres images en disant "il n'y a pas de chat", puis tester si en donnant une image de chat on nous répond "il y a un chat" et inversement.

2.2.2 Machines à vecteurs de support :

On se place dans le cadre d'un apprentissage supervisé. Chaque exemple à apprendre est représenté par un point dans l'espace à n dimensions des caractéristiques mesurées.

L'objectif est de séparer les points appartenant à l'une ou l'autre des catégories. On dit qu'ils sont séparables linéairement s'il existe un hyperplan délimitant deux demi-espaces ne contenant chacun que les points d'une seule catégorie.

Si les données sont séparables linéairement on calcule l'hyperplan séparateur maximisant la distance minimale aux points des deux catégories, on a ainsi de meilleurs résultats en catégorisant par la suite un nouveau point.

Si les données ne sont pas séparables linéairement, on peut presque toujours trouver un espace, dit de description, de bien plus grande dimension dans lequel on les injecte, les rendant ainsi linéairement séparables. On va utiliser pour cela des fonctions nommées fonctions noyau pour changer d'espace puis utiliser la méthode précédente.

Après l'apprentissage, lorsque l'on veut classer un nouvel élément, on le place dans l'espace précédent et on répond en fonction de sa position par rapport à l'hyperplan.

Cette méthode est également appelée séparateur à vastes marges ; elle sera notée SVM dans la suite de ce rapport. J'ai choisie de l'utiliser car elle est connue pour être efficace, générale et simple dans son implémentation. Sa complexité est quadratique dans la bibliothèque scikit-learn, que j'ai utilisée durant mon stage.

3 Travaux personnels

3.1 Idées générales

Mon but est d'utiliser la transformée de Fourier pour essayer de rendre l'apprentissage avec la méthode SVM plus efficace.

J'ai principalement travaillé avec des enchainements d'images pour reconnaître certaines formes quelle que soit leur position dans l'image. L'utilisation d'images permet la reconnaissance (et donc la vérification) à l'œil nu des schémas récurrents, contrairement à des structures irrégulières et plus exotiques.

Pour toute image on peut définir un graphe associé de la façon suivante :

- chaque sommet correspond à un pixel.
- le signal correspond aux couleurs prises par les pixels.
- les arêtes relient les sommets associés à des pixels voisins.

3.2 Premiers tests

J'ai tout d'abord pris la décision d'utiliser le langage Python, car c'est un langage que je connaissais et qui permet l'utilisation de différentes librairies adaptées à ces problématiques.

3.2.1 Inversion de matrices

J'avais besoin de savoir les temps nécessaires pour inverser une matrice (opération utilisée lors de la transformation de Fourier), afin de savoir avec quelles tailles de données je pouvais raisonnablement travailler.

J'ai effectué des tests et obtenu les temps suivants :

taille d'un coté	1000	2000	3000	4000	5000
temps pour inverser	5s	40s	2min	5,5min	10.5 min

3.2.2 Premières transformées et observations

J'ai ensuite réalisé mes premières transformées de Fourier. Je les ai affichées sous différentes formes, notamment un diagramme de fréquences, qui au sein du laboratoire où j'effectuais mon stage a pris le nom officiel d'"*emmagramme*". J'ai initialement travaillé avec des données d'encéphalogrammes, mais je suis rapidement passée à un premier jeu d'images.

A ce moment là j'ai travaillé avec Bastien Pasdeloup et utilisé MATLAB.

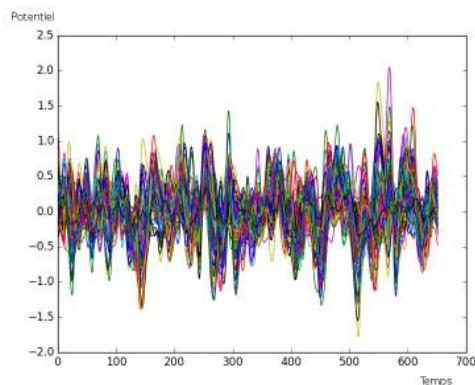


FIGURE 2 – Superposition des signaux d'encéphalogrammes

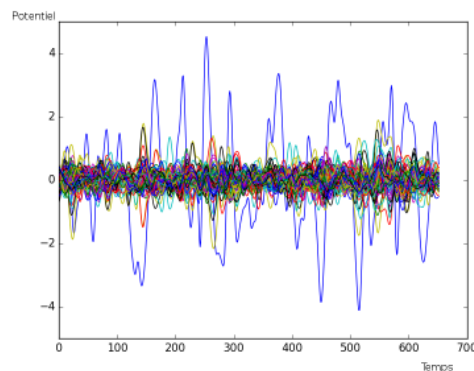


FIGURE 3 – Superposition des transformées de Fourier des signaux

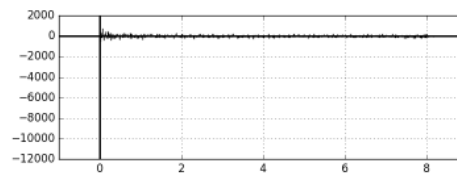
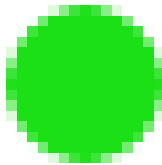


FIGURE 5 – "Spectrogramme" associé

FIGURE 4 – Première forme sur laquelle je faisais des transformées de Fourier, pour diverses positions dans l'image

Je cherchais alors à observer des motifs récurrents dans les transformées de Fourier. Mais à l'œil nu rien ne paraissait significatif.

3.3 Premiers apprentissages

L'étape suivante a été de voir si la transformée de Fourier pouvait être utile en comparant les résultats d'un apprentissage SVM sur les données initiales et sur leur transformée de Fourier.

J'ai à nouveau travaillé sur des images ; les résultats les plus significatifs (ceux où l'apprentissage se fait avec un nombre non négligeable d'exemple) correspondent à la distinction d'un disque et d'un carré (pour différentes positions dans l'image). Le set de données d'apprentissage a été généré en décalant le motif de base d'image en image.

J'ai comparé le nombre d'exemples d'apprentissages utiles pour être efficace en apprenant avec ou sans transformée de Fourier. Les transformées de Fourier nous permettent d'utiliser moins d'exemples d'apprentissage. Dans le cas disques et carrés (cf figure 6), il faut 40 exemples de transformées de Fourier pour 80 exemples de données de base.

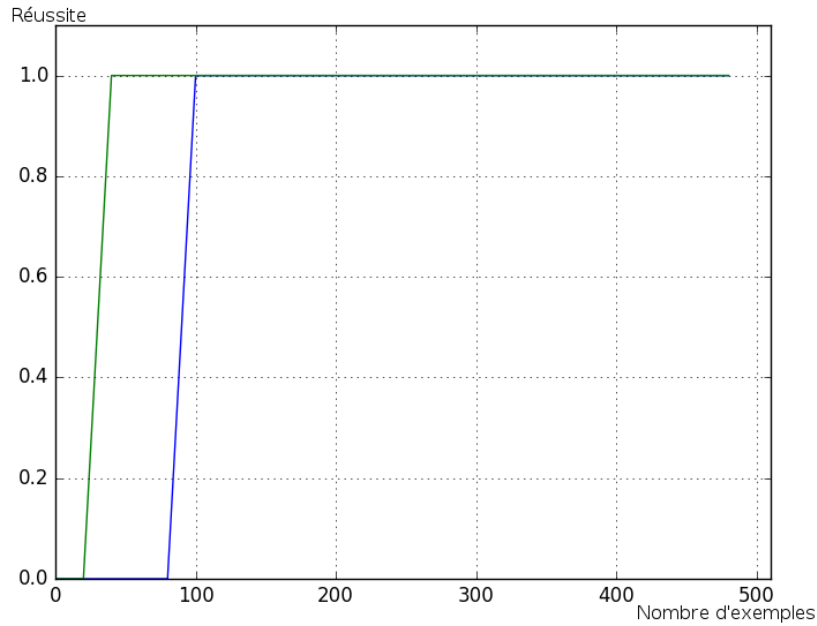


FIGURE 6 – Réussite de l'apprentissage en fonction du nombre d'exemples d'apprentissage, pour l'apprentissage direct (bleu) et l'apprentissage de transformées de Fourier (vert).

3.4 Transformation de transformées

On se pose la question de l'impact de modifications sur les transformées de Fourier. Deux transformées de Fourier proches ou ayant des caractéristiques communes correspondent-elles à des images similaires ?

3.4.1 Modification de valeurs

J'ai tout d'abord ajouté du bruit (ajout d'une matrice aléatoire de loi uniforme générée via Python) à la transformée d'une image avant de reconstituer celle-ci. On obtient la même image sur laquelle s'ajoute des traces allongées cf figure 7 et 8 (contrairement au bruitage direct qui ne donne que des points supplémentaires figure 9).

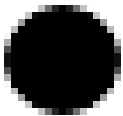


FIGURE 7 – Image de départ sur laquelle on va effectuer des modifications

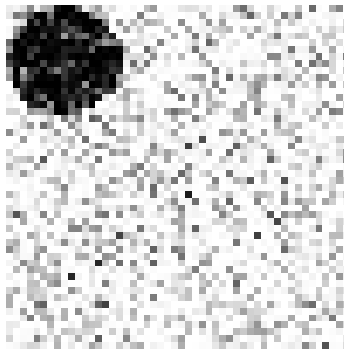


FIGURE 8 – Image bruitée directement sans utiliser la transformée



FIGURE 9 – Image reconstituée à partir de la transformée bruitée

Il n'en sort rien de significatif et pouvant être utilisé.

3.4.2 Suppression de valeurs

On sait que la majeure partie de l'information est codée sur les premières valeurs des transformées de Fourier, j'ai donc voulu observer ce qu'il se passait si l'on supprimait les dernières valeurs puis recréait une image.



FIGURE 10 – Image de départ

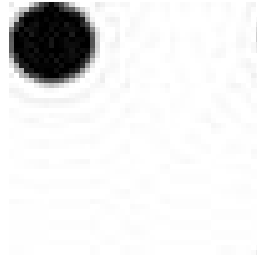


FIGURE 11 – On conserve 1000 valeurs

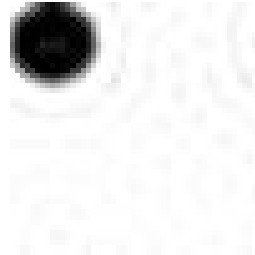


FIGURE 12 – On conserve 500 valeurs

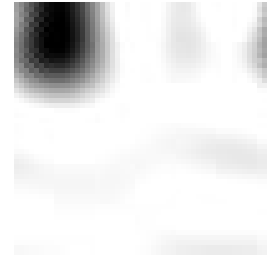


FIGURE 13 – On conserve 100 valeurs



FIGURE 14 – Image de départ

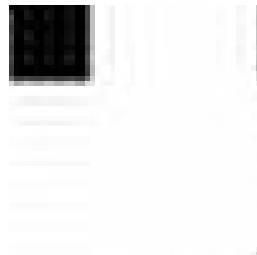


FIGURE 15 – On conserve 1000 valeurs

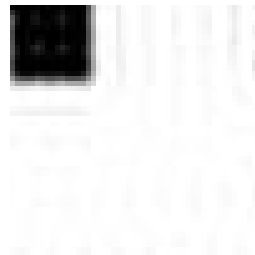


FIGURE 16 – On conserve 500 valeurs

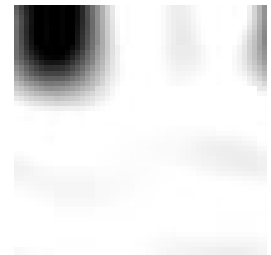


FIGURE 17 – On conserve 100 valeurs

Il s'avère que la suppression de valeurs n'empêche pas la reconnaissance de l'image, cependant plus on en supprime plus celle-ci devient floue.

On voit ici l'évolution d'un disque (10, 11, 12, 13) et d'un carré (14, 15, 16 et 17), ceux-ci peuvent encore être facilement différenciés alors qu'on ne conserve que 500 valeurs sur les 2500 de base (image en 50x50).

3.5 Apprentissage de transformées simplifiées

L'utilisation de transformées que l'on a simplifiées en supprimant des valeurs peut être intéressant puisqu'il permet de traiter des sets de données de tailles moins importantes et donc de simplifier les calculs. J'ai donc étudié l'utilisation de tels sets simplifiés, en observant à partir de combien de valeurs on apprenait encore efficacement pour différents taille de set d'exemples.

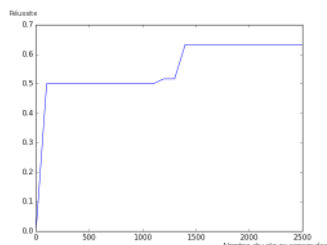


FIGURE 18 – Apprentissage pour un set de 10 exemples

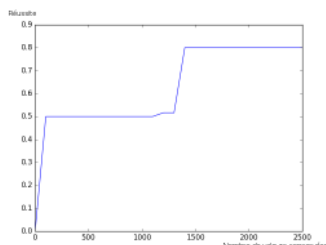


FIGURE 19 – Apprentissage pour un set de 20 exemples

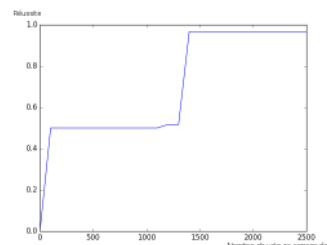


FIGURE 20 – Apprentissage pour un set de 30 exemples

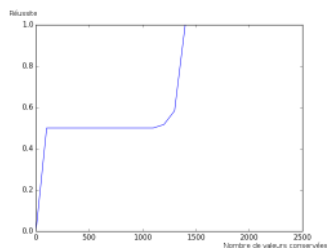


FIGURE 21 – Apprentissage pour un set de 40 exemples

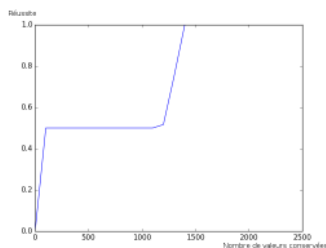


FIGURE 22 – Apprentissage pour un set de 50 exemples

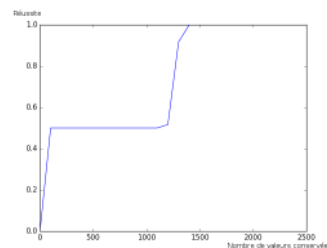


FIGURE 23 – Apprentissage pour un set de 60 exemples

Dans le cas des disques/carrés, on sait que l'apprentissage avec une transformée de Fourier totale est optimal pour des sets d'exemples supérieurs à 40. On constate ici que pour des apprentissages avec des set de taille supérieurs à 40 (cf images 21, 22 et 23), les courbes sont similaires et on arrive à un plateau à 1 i.e. 100% de réussite. Pour des sets supérieurs à 40, on ne gagne donc rien à augmenter le nombre d'exemples d'apprentissage. Pour les sets inférieurs à 40 (cf images 18, 19 et 20), on observe qu'on n'atteint pas la réussite totale même si on est de plus en plus efficace.

Les plateaux à 0.5 correspondent donc à une erreur une fois sur deux ce qui s'explique par le fait qu'on reconnaît toujours des ronds alors qu'il y a un carré une fois sur deux. On n'est donc efficace pour 40 exemples où l'on conserve 1400 valeurs.

3.6 Comparaison de temps

J'ai ensuite comparé les temps d'apprentissage¹ pour un apprentissage direct et un apprentissage de transformées de Fourier simplifiées suivi de 60 tests (24, 25, 26 et 27). On voit malheureusement qu'effectuer les transformations est plus coûteux. En effet l'usage de transformation ne devient plus efficace qu'à partir d'un nombre d'exemples d'apprentissage inutilement grand (par exemple 500 exemples pour 1250 valeurs conservées).

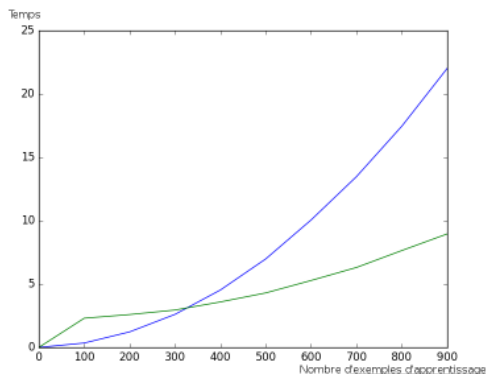


FIGURE 24 – Temps d'apprentissage pour 750 valeurs conservées

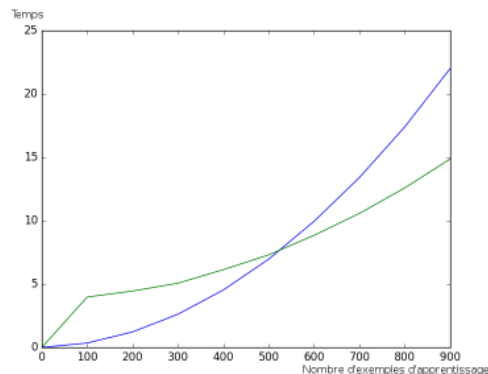


FIGURE 25 – Temps d'apprentissage pour 1250 valeurs conservées

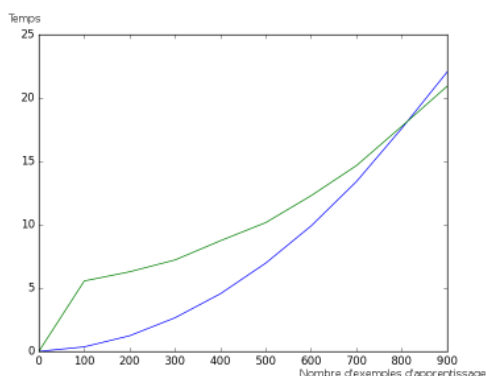


FIGURE 26 – Temps d'apprentissage pour 1750 valeurs conservées

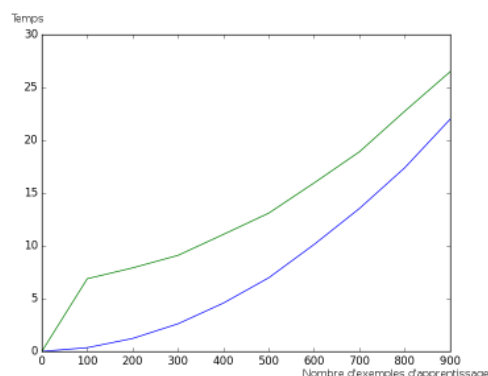


FIGURE 27 – Temps d'apprentissage pour 2250 valeurs conservées

On atteint 100% de réussite en apprenant 40 exemples de transformées de Fourier dont on garde 1400 valeurs, ou 80 exemples sans utilisation de la transformée.

J'ai donc comparé les temps de calcul pour ces deux cas (cf image 28). On conclut donc qu'il est plus avantageux en temps d'utiliser des transformées de Fourier simplifiées si les calculs de simplifications sont déjà fait.

1. Toutes les mesures de temps sont réalisées à l'aide d'un Intel Core i7-3610QM (8Go de RAM) sous Linux Mint 17.3 (python 2.7).

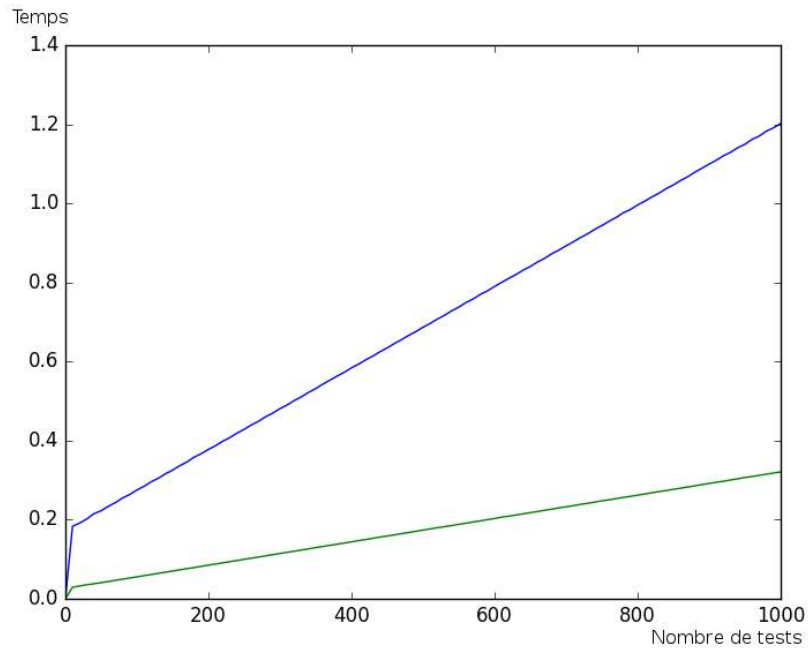


FIGURE 28 – Temps d’apprentissage pour différents nombres de tests pour 40 exemples de transformées de Fourier avec 1400 valeurs conservées (en vert) et 80 exemples de données directes (en bleu)

3.7 Détection d’un motif parmi plusieurs

Je me suis intéressée ensuite à la détection d’un motif sur une image en contenant plusieurs. Cette problématique est particulièrement utile, car on cherche souvent à détecter un motif indépendamment des autres perturbations. J’ai pour cela ajouté du bruit aux images. On observe (cf image 29) que l’apprentissage via la transformée de Fourier est plus rapide que l’apprentissage direct. En fait, ces résultats sont similaires à ceux des tests précédents, où l’on apprenait les motifs seuls, mais il faut ici davantage d’exemples pour apprendre avec un taux de succès de 1.

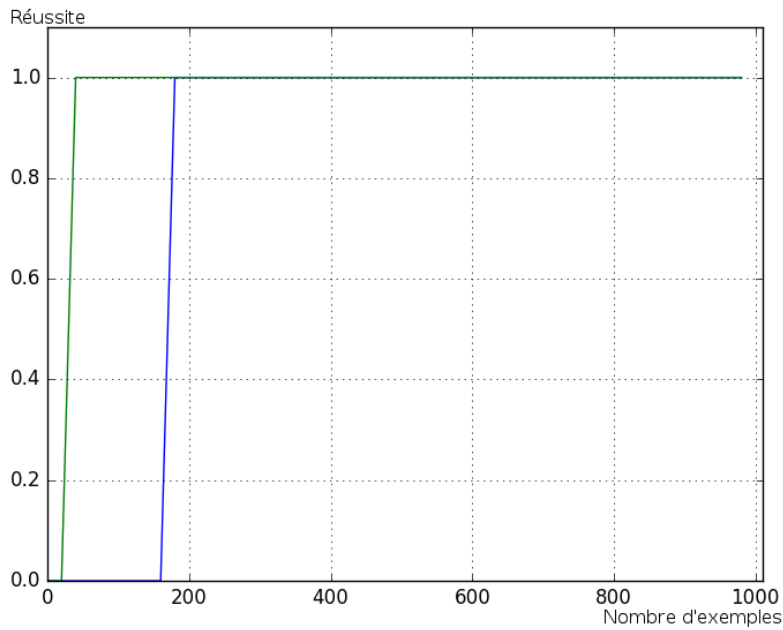


FIGURE 29 – Résultats pour de l'apprentissage direct et de transformées de Fourier en fonction du nombre d'exemples.

3.8 Divagations

Au cours de mon stage j'ai également effectué des tests annexes qui ne s'inscrivent pas directement dans la continuité de la détection de motifs récurrent. J'ai notamment étudié l'influence de la saturation des couleurs (pour diverses valeurs de changement) sur la qualité de l'apprentissage d'une part et la transformée de Fourier de graphes d'autre part. Aucun résultat n'a été concluant.

Je me suis aussi penchée sur l'apprentissage des positions et non des formes au sein d'une image : il fallait alors conserver presque toutes les valeurs des transformées de Fourier et utiliser dans ce cas un nombre supérieur d'exemples que pour l'apprentissage direct.

4 Conclusion

Le but de ce stage était d'essayer d'utiliser la transformée de Fourier pour détecter des motifs dans signaux sur graphes. Il semble que l'utilisation des transformées de Fourier des valeurs à la place des données de base améliore la classification par la méthode SVM. De plus, il n'est pas nécessaire de conserver toutes les valeurs de la transformée de Fourier : il suffit alors travailler avec moins de données, ce qui est un avantage considérable.

Je n'ai pas eu le temps de plus approfondir la reconnaissance de schémas au sein d'images contenant d'autres motifs, alors qu'il s'agissait de tout l'intérêt de la démarche. Dans des travaux futurs, on pourrait par exemple ajouter des formes de même ampleur et non juste du bruit. On peut également revoir le sujet en cherchant une moins bonne précision. Je n'ai traité que des images et des images de taille restreintes, il serait intéressant d'appliquer cela à d'autres types de données comme des images plus grandes ou issues de domaines irréguliers.

Références

- [1] Aliaksei Sandryhaila and José M. F. Moura. Discrete signal processing on graphs : Frequency analysis. *IEEE Trans. Signal Processing*, 62(12) :3042–3054, 2014.
- [2] David I. Shuman, Sunil K. Narang, Pascal Frossard, Antonio Ortega, and Pierre Vandergheynst. The emerging field of signal processing on graphs : Extending high-dimensional data analysis to networks and other irregular domains. *IEEE Signal Process. Mag.*, 30(3) :83–98, 2013.